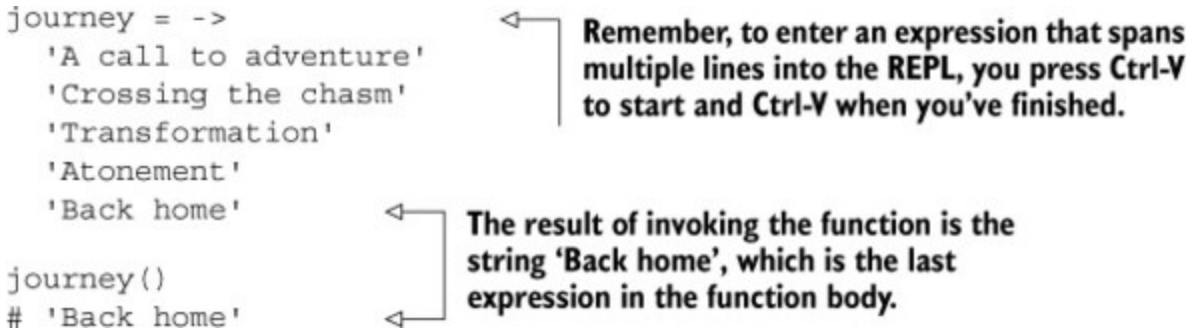
When this function is invoked, it evaluates the expressions in the body of the function:

```
threeTimesFour()
# 12
```

The *last* expression evaluated inside the function is the evaluation of the function itself:



That `threeTimesFour` function isn't very useful, though. How do you multiply *any* two numbers? By using function parameters:



Multiplication is rather boring, though, especially when there's already an operator to do it for you. What are functions good for then? The *real* power of functions starts when you use them to define new things that don't have operators. Why? Because when you define things that don't have operators, you create your own language.

3.1.2. Custom operations

Consider a function to covert gigabytes to megabytes:

```
gigabytesToMegabytes = (gigabytes) -> gigabytes * 1024
```

This function defines an operation that isn't built into the language. Try it: